

```

        .fetch_one(&mut transaction).await?;
    transaction.commit().await?;
    Ok(person)
}

```

You can modify your main fetch to use it:

```

#[tokio::main]
async fn main() -> Result<()> {
    let db = DBApplication::new("postgres://
postgres:postgres@localhost/sqlx-demo".into()).await?;
    let sibi = db.create_person(NewPerson{ name: "Sibi".
into(), age: 20, email: "test@psibi.in".into()}).await?;
    println!("Sibi: {:?}", sibi);
    Ok(())
}

```

Now let's check if the function returns early because of an error, if the transaction rolls back. We will simulate a failure in the above function by an early return after insert:

```

pub async fn create_person(&self, person: NewPerson) ->
Result<Person> {
    let mut transaction = self.pool.begin().await?;
    let person = sqlx::query_as("INSERT INTO person(name,
age, email) VALUES ($1, $2, $3) RETURNING *;")
        .bind(person.name)
        .bind(person.age)
        .bind(person.email)
        .fetch_one(&mut transaction).await?;
    bail!("Simulate error");
    transaction.commit().await?;
    Ok(person)
}

```

And the main function will be like this:

```

#[tokio::main]
async fn main() -> Result<()> {
    let db = DBApplication::new("postgres://
postgres:postgres@localhost/sqlx-demo".into()).await?;
    let sibi = db.create_person(NewPerson{ name: "Sibi".
into(), age: 20, email: "test@psibi.in".into()}).await?;
    println!("Sibi: {:?}", sibi);
    Ok(())
}

```

Before running the executable, you can check the current rows in your table:

```

sqlx-demo=# select * from person;
 id | name | age | email
----+-----+-----+-----
(0 rows)

```

Now let's execute the program:

```

❏ cargo run    Finished dev [unoptimized + debuginfo]
target(s) in 0.05s
    Running `target/debug/sqlx-demo`
Error: Simulate error

```

You can see that it did return an error. You can also verify that the transaction has rolled back and has not inserted the row, by inspecting the rows:

```

sqlx-demo=# select * from person;
 id | name | age | email
----+-----+-----+-----
(0 rows)

```

Other CRUD operations

Similarly, you can implement other CRUD (create, read, update, delete) operations too:

```

pub async fn get_person(&self, id: i64) -> Result<Person> {
    let mut transaction = self.pool.begin().await?;
    let person = sqlx::query_as("SELECT * from person where
id = $1")
        .bind(id)
        .fetch_one(&mut transaction).await?;
    transaction.commit().await?;
    Ok(person)
}

```

```

pub async fn update_person_name(&self, id: i64, name: String)
-> Result<Person> {
    let mut transaction = self.pool.begin().await?;
    let person = sqlx::query_as("UPDATE person SET name = $1
WHERE id = $2 RETURNING *")
        .bind(name)
        .bind(id)
        .fetch_one(&mut transaction).await?;
    transaction.commit().await?;
    Ok(person)
}

```

```

pub async fn delete_book(&self, id: i64) -> Result<()> {
    let mut transaction = self.pool.begin().await?;
    sqlx::query("DELETE FROM book WHERE id = $1")

```